High-Performance Pandas: eval() and query()

Python Data Science Handbook — Chapter 3, Section 3.12

Killing the temporary arrays — how string-based expressions let Pandas fuse a compound calculation into a single pass.

1 The problem: vectorization is fast, but greedy

By now you have absorbed the central performance lesson of NumPy and Pandas: *vectorize everything*. Replacing a Python `for` loop with an array expression like `a + b` hands the work to compiled C code, and you typically see one or two *orders of magnitude* speed-up. So you would be forgiven for thinking that, once an expression is written in array form, there is nothing left to optimize.

There is. The issue hides inside *compound* expressions — expressions built from several operators, like

$$\texttt{result} \;=\; (A + B) * (C - D).$$

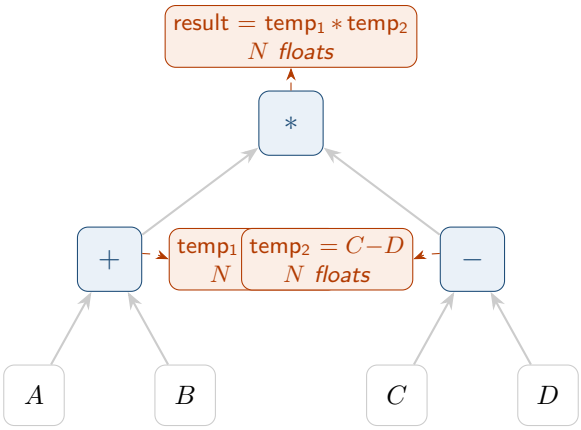
Each binary operator (`+`, `-`, `*`) is its own vectorized call, and each one produces a brand-new array. The Python interpreter evaluates the expression one operator at a time, and it has no way to know that you only care about the final answer. So it dutifully *materializes* every intermediate result in memory.

Background & Context

Why a fresh array per operator? NumPy operators are ordinary function calls under the hood: `A + B` is really `A.__add__(B)`, and a function has to *return* something. The only thing it can return is a freshly allocated array holding $A_i + B_i$ for every i. The interpreter then feeds that array into the next operator, which again allocates. Nobody in this chain is allowed to "peek ahead" and reuse memory, because each call is evaluated in isolation. The result is correct — it is just wasteful.

1.1 Counting the waste

Consider `(A + B) * (C - D)` on four arrays of N elements each. Python evaluates it as three steps, and allocates a temporary of size N at each internal node of the expression tree:



Three internal nodes ⇒ **three** full-length temporary arrays allocated.
Only the top one is the answer you wanted; the other two are pure overhead.

For small N this is invisible. But when N is large the temporaries dominate, and the reason is not really the allocation *count* — it is the *memory traffic*.

Key Idea

Modern CPUs are starved not for arithmetic but for *data*. A floating-point multiply is nearly free; fetching the operands from main memory is the expensive part. The CPU has a small, fast **cache** (a few MB). If an array fits in cache, repeated passes over it are cheap. Each temporary that does *not* fit forces the data to be streamed out to RAM and back, and that round trip — the **memory bandwidth** bottleneck — is what actually costs you. Allocating three N-element temporaries means writing and re-reading roughly $3N$ values through that bottleneck.

2 The fix: numexpr and element-wise fusion

The cure is to evaluate the whole expression *element by element*, computing the final `result[i]` for each `i` before moving on — never building the big intermediate arrays at all. For each index i you compute $(A_i + B_i)(C_i - D_i)$ using a couple of scalar temporaries, store one number, and move on. The only N-element array that ever exists is the answer.

This is exactly what the **numexpr** library does. You hand it the expression as a *string*, and it compiles a little plan that walks all the arrays in lockstep in one fused pass. As a bonus, because the work is so regular, numexpr can split it across multiple CPU threads.

Naive: 3 passes

A+B → temp1

C-D → temp2

temp1*temp2 → out

extra memory: **2** big temps
traffic: $\sim 3N$ writes

Fused (numexpr): 1 pass

for each i:
out[i] = (A[i]+B[i])*(C[i]-D[i])

extra memory: **0** big temps
traffic: $\sim N$ writes

Pandas wraps numexpr in three friendly entry points: the top-level **`pd.eval`**, the method `DataFrame.eval`, and the method `DataFrame.query`. We will take them in turn.

Python Refresher

A quick reminder on *bitwise* versus *logical* operators, because they matter below. On NumPy/-Pandas boolean arrays you must use the bitwise operators `&` (and) and `|` (or), element by element. The Python keywords **`and`**/**`or`** do *not* work on arrays — they try to collapse the whole array to a single **`True`**/**`False`** and raise an error. Inside an **`eval`**/`query` string, however, you may write the *words* **`and`** and **`or`**, and Pandas maps them to the elementwise versions for you.

3 pd.eval: evaluating a string across whole objects

`pd.eval` takes a string, parses it, and evaluates it with numexpr. Suppose we build four large DataFrames:

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(0)
nrows, ncols = 1_000_000, 4
A, B, C, D = (pd.DataFrame(rng.random((nrows, ncols))) for _ in range(4))
```

The ordinary way and the **`eval`** way give identical numbers:

```
direct = (A + B) * (C - D)              # 3 temporaries
fused  = pd.eval('(A + B) * (C - D)')   # numexpr, no big temporaries
np.allclose(direct, fused)
```

```
True
```

The two results agree, but the second never materialized `temp1` or `temp2`. The variable names inside the string (`A`, `B`, `C`, `D`) are looked up in the surrounding Python namespace, so the expression reads just like the real one.

Intuition

Think of **`pd.eval`** as handing Pandas a *recipe* (the string) instead of forcing it to cook each step and plate the result before reading the next instruction. With the recipe in hand, Pandas can see the whole dish at once and skip dirtying every pan along the way.

3.1 What you can and cannot put in the string

numexpr is fast precisely because it supports a *restricted* mini-language, not arbitrary Python. Inside a **`pd.eval`** string you may use:

- **Arithmetic:** `+`, `-`, `*`, `/`, and even `**` and `%`.
- **Comparisons:** `<`, $\leq$, `==`, `!=`, $\geq$, `>`, including chained forms like `a < b` $\leq$`c`.
- **Boolean logic:** the bitwise `&` and `|`, *and* the literal words **`and`**/**`or`**.

- **Attribute access and indexing:** `df.A`, `arr[0]`.

What you may *not* do is call arbitrary functions, run loops, comprehensions, or conditionals. There is no `np.sqrt(...)` of your choosing, no `for`, no `if/else`. If you need those, fall back to ordinary syntax — numexpr's whole bargain is that it only handles the simple, regular operations it can fuse and thread.

Common Pitfall

`pd.eval` compares *aligned* objects by name, but it does *not* replace ordinary Pandas alignment subtleties. If `A` and `B` have mismatched indices, you still get `NaN` where they fail to line up — `eval` changes *how fast* the arithmetic runs, not *what* arithmetic means. Always sanity-check with `np.allclose` against the direct expression the first time you convert a hot path.

4 `DataFrame.eval`: column names become variables

`pd.eval` is global; you pass full objects by name. Far more ergonomic for day-to-day work is the *method* form, `DataFrame.eval`, which evaluates an expression *within the context of one DataFrame*. The payoff: bare **column names** act as variables.

```
df = pd.DataFrame(rng.random((1_000_000, 3)), columns=['A', 'B', 'C'])

# 'A', 'B', 'C' refer to columns -- no df['...'] noise
result = df.eval('(A + B) / (C - 1)')
```

Compare that to the equivalent direct version, `(df['A'] + df['B'])/ (df['C'] - 1)`, and you can see why `eval` is easier to read: the algebra is right there, undiluted by indexing brackets.

4.1 Reaching outside: the `@` prefix

Sometimes you want to mix a column with a plain Python variable. Inside the string, a bare name is assumed to be a *column*, so to point at a local Python variable instead you prefix it with `@`:

```
offset = 0.5
df.eval('A + @offset')   # column A plus the Python scalar offset
```

Key Idea

The `@` symbol is the namespace switch. **Bare name = a column of this DataFrame; `@name` = a variable from the surrounding Python scope.** Without it, `eval` would look for a column literally named "offset," not find one, and raise an error. This same rule applies in `query` below.

4.2 Creating and modifying columns in place

`DataFrame.eval` can also *assign*. Putting a name and `=` on the left creates (or overwrites) a column:

```
df.eval('D = (A + B) / C', inplace=True)
df.head(3)
```

```
    A       B       C       D
0   0.6369   0.2697   0.0409   22.654
1   0.0165   0.8132   0.9127   0.9089
2   0.6066   0.7295   0.5436   2.4576
```

With `inplace=True` the new column `D` is written straight into `df`; with `inplace=False` (the default) you get back a *copy* carrying the extra column and leave the original untouched. You can also *overwrite* an existing column the same way, e.g. `df.eval('D = D + 1', inplace=True)`.

5 DataFrame.query: filtering rows cleanly

`eval` computes new *values*. The closely related `query` method selects *rows* — it is for filtering. The classic boolean-mask idiom for "rows where A is small and B is large" looks like this:

```
mask_form = df[(df.A < 0.5) & (df.B > 0.5)]
```

That single line quietly builds several temporaries: the boolean array `df.A < 0.5`, the boolean array `df.B > 0.5`, and their combined `&`. `query` expresses the same filter as one string and lets numexpr do it in a fused pass:

```
query_form = df.query('A < 0.5 and B > 0.5')
np.allclose(mask_form, query_form)
```

```
True
```

Notice we wrote the word `and` rather than `&` — inside the string both are allowed, and the word reads more naturally. `query` understands the same `@` convention for local variables:

```
cutoff = 0.5
df.query('A < @cutoff and B > @cutoff')
```

Intuition

A useful way to remember the division of labor: **`eval` makes columns, `query` keeps rows.** If the result you want is a new Series/column of computed values, reach for `eval`; if the result you want is a subset of the existing rows, reach for `query`. Both run through the same numexpr engine, so both dodge the temporaries that the equivalent bracket syntax would spawn.

6 When is it actually worth it?

Here is the part people skip, and then misuse these tools. The fused-evaluation trick pays for itself only when the *memory savings* outweigh the *fixed overhead* of parsing a string and dispatching to numexpr. That fixed cost is small but not zero, and it does not shrink with your data.

Common Pitfall

On small data, `eval` and `query` are no faster — often slower. For a DataFrame of a few hundred rows, the intermediate "temporaries" fit comfortably in cache, so building them costs almost nothing, while the string-parsing and function-call overhead of `eval`/`query` is now a relatively large tax. You can spend more time setting up the fused evaluation than you would have spent just doing the arithmetic. Do not sprinkle `eval` everywhere "for speed" — on small frames it is pure ceremony.

The break-even point is roughly when your arrays stop fitting in the CPU cache — think tens of MB and up. A cheap way to build intuition is to look at the raw size of the data the expression touches:

```
df.values.nbytes    # bytes backing the DataFrame
```

```
24000000
```

If `df.values.nbytes` is in the megabytes-and-beyond range (comfortably larger than your CPU's cache), the temporaries genuinely hurt and `eval`/`query` can win on both speed *and* memory. If it is a few kilobytes, you are well inside cache and the ordinary syntax will be at least as fast.

Key Idea

Two independent reasons to reach for these tools, and they apply at different scales:

- **Memory / speed** — only on *large* data (data $\gtrsim$ cache, roughly MBs+), where avoiding temporaries cuts memory traffic and peak RAM.
- **Readability** — at *any* scale, `df.query('A < 0.5 and B > 0.5')` is simply clearer than the bracketed mask. On small data, use them *only* for this reason, accepting that they are not faster.

Section recap

- Compound array expressions like `(A + B)* (C - D)` allocate one full-length *temporary* per internal operator. For large data those temporaries blow past the CPU cache and the cost is *memory bandwidth*, not arithmetic.
- **numexpr** evaluates the whole expression element-by-element in a single fused (and optionally multi-threaded) pass, so the only big array created is the answer.
- `pd.eval('...')` runs a string expression across whole objects; supports arithmetic, comparisons, bitwise `&`/`|` and the words `and`/`or`, attribute access, and indexing — *not* arbitrary function calls or loops.
- `DataFrame.eval('...')` treats bare column names as variables, uses `@name` for local Python variables, and can create/overwrite columns (`df.eval('D = (A + B)/ C', inplace=True)`).
- `DataFrame.query('...')` filters rows and is both cleaner and more memory-efficient than the `df[(df.A < 0.5)& (df.B > 0.5)]` mask form; it also supports `@`.
- **Worth it only on large data** (check `df.values.nbytes` vs. cache). On small frames the overhead makes them no faster — keep them there purely for readability.